CHAUDHARY CHARAN SINGH UNIVERSITY (CCSU) • BCA V SEMESTER

BCA-501: Introduction to DBMS

Session: December 2025 Examination | **Paper Code:** 18021 | **Authentic University Solved Paper (All Questions Solved)** | **Time:** 3 Hours • **Max Marks:** 75

SECTION A — VERY SHORT ANSWER QUESTIONS (5 × 3 = 15 Marks)

Q1 (3M) What is a Database?

Definition: A **Database** is an organized, logically coherent collection of related data modeled to represent real-world entities and enterprise operations. Key properties include:

- **Self-describing nature:** Stores both data and metadata (system catalog/data dictionary).
- **Program-data independence:** Structure of data files is stored separately from user access programs.
- **Multi-user transaction processing:** Guarantees concurrent access without collision or corruption.

Q2 (3M) State the key advantages of DBMS over File Processing Systems.

Advantages:

Aspect	Traditional File System	Database Management System (DBMS)
Data Redundancy	High redundancy across duplicate files	Controlled and minimized via normalization
Data Inconsistency	High risk due to isolated updates	Eliminated via centralized repository
Data Independence	None; changes in files break code	Physical and Logical Data Independence achieved
Concurrent Access	Prone to race conditions and lost updates	ACID compliant concurrency control (2PL, timestamp)
Security & Integrity	OS-level passwords, hard to enforce	Fine-grained role-based access & constraints

---

Q3 (3M) Who is a Database Administrator (DBA)? Mention key duties.

Definition: A **Database Administrator (DBA)** is the central authority responsible for supervising and administering the entire database environment.
Primary Functions:

1. **Schema Definition:** Designing and altering conceptual, external, and internal schemas.
2. **Storage & Access Tuning:** Specifying storage structures, file organizations, and primary/secondary indexes.
3. **Security & Authorization:** Granting and revoking user privileges, managing roles, and enforcing access control.
4. **Backup & Recovery:** Scheduling periodic backups and executing disaster recovery strategies.
5. **Integrity Enforcement:** Defining domain, entity, and referential integrity constraints.

Q4 (3M) Differentiate between Schema and Instance in DBMS.

Comparison:

- **Database Schema:** The overall logical blueprint, structure, and design of the database (e.g., table names, column data types, constraints). It is specified during system design and changes very infrequently.
- **Database Instance (State):** The actual data content populated within the database at a specific moment in time. It changes dynamically whenever records are inserted, updated, or deleted.

Analogy: In programming languages, a Schema corresponds to a Class/Data Type, whereas an Instance corresponds to an Object/Value of that type.

Q5 (3M) **What do you mean by Data Inconsistency in File Management Systems? Explain.**

Explanation: Data Inconsistency occurs when multiple copies of the exact same data across different files contain conflicting, contradictory values.

Example: In a university file system, a student's address is recorded in the *Library File* and also in the *Fee Accounting File*. If the student changes their residential address and only the Library office updates their local file while the Fee office does not, the university now holds two contradictory addresses for the same student. This occurs due to uncoordinated duplicate storage without centralized integrity control.

SECTION B — SHORT ANSWER QUESTIONS (2 × 7.5 = 15 Marks)

Q6 (7.5M) **Explain ER Diagram. Describe all standard ER notations and symbols with diagrams.**

Definition: An **Entity-Relationship (ER) Diagram** is a graphical semantic modeling tool that represents the logical structure of a database in terms of entities, their attributes, and relationships.

Standard Symbols & Notations:

```
[ Rectangle ]           → Entity Set (e.g., STUDENT, COURSE)
[[ Double Rectangle ]]  → Weak Entity Set (e.g., DEPENDENT, INSTALLMENT)
( Ellipse / Oval )      → Attribute (e.g., Age, Name)
( Underlined Text )     → Primary Key Attribute (e.g., RollNo)
(( Double Ellipse ))    → Multivalued Attribute (e.g., PhoneNumbers, Skills)
( Dashed Ellipse )      → Derived Attribute (e.g., Age calculated from DOB)
{ Diamond }             → Relationship Set (e.g., ENROLLED_IN)
{{ Double Diamond }}    → Identifying Relationship for Weak Entity Set
======== Double Line    → Total Participation (Every entity must participate)
-------- Single Line    → Partial Participation
```

Q7 (7.5M) **Explain Primary Key, Foreign Key, and Candidate Key with suitable SQL examples.**

1. Candidate Key: A minimal super key — an attribute or minimal set of attributes that uniquely identifies each tuple in a relation without any redundant attributes.

2. Primary Key: The specific candidate key chosen by the database designer to uniquely identify tuples in a table. It cannot accept duplicate values and cannot contain `NULL`.

3. Foreign Key: An attribute in a child table whose values must match the primary key of a referenced parent table (or be `NULL`), enforcing referential integrity.

SQL Implementation Example:

```sql
-- Parent Table: DEPARTMENT
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,              -- Primary Key
    DeptName VARCHAR(50) NOT NULL UNIQUE -- Candidate Key
);

-- Child Table: EMPLOYEE
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,               -- Primary Key
    EmpName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,           -- Candidate Key
    DeptID INT,                          -- Foreign Key
    CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Q8 (7.5M) **Differentiate between DDL and DML commands with syntax and examples.**

Comparison:

Feature	Data Definition Language (DDL)	Data Manipulation Language (DML)
Purpose	Defines and modifies the schema structure of the database.	Manages, accesses, and manipulates the data within the tables.
Commit Nature	Auto-committed permanently (cannot be rolled back in most RDBMS).	Transactional (can be committed or rolled back using TCL).
Commands	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	`INSERT`, `UPDATE`, `DELETE`, `SELECT`

SQL Examples:

```
-- DDL Commands:
CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50));
ALTER TABLE Student ADD COLUMN City VARCHAR(30);

-- DML Commands:
INSERT INTO Student (RollNo, Name, City) VALUES (101, 'Aman', 'Meerut');
UPDATE Student SET City = 'Delhi' WHERE RollNo = 101;
```

SECTION C — DETAILED ANSWER QUESTIONS (3 × 15 = 45 Marks)

Q9 (15M) **Explain the ACID properties of a transaction with practical examples. Why are they essential?**

Definition: A **Transaction** is a logical unit of database processing that includes one or more database access operations. To preserve database consistency and reliability, every DBMS must guarantee the four **ACID Properties**:

1. Atomicity ("All or Nothing"):

- Either all operations of the transaction are committed successfully, or none take effect. If failure occurs mid-transaction, all changes are rolled back.
- *Example:* Bank transfer of \$500 from Account A to Account B. If \$500 is debited from A and power fails before crediting B, the recovery subsystem rolls back A's debit.

2. Consistency (Preservation of Invariants):

- Execution of a transaction in isolation preserves database integrity constraints (e.g., total balance of A + B must remain unchanged before and after the transfer).

3. Isolation (Independence of Concurrent Executions):

- Even if multiple transactions execute concurrently, the intermediate uncommitted state of one transaction is hidden from others, producing a serializable execution schedule. Enforced via locking (2PL) and timestamp ordering.

4. Durability (Survivability of Committed Data):

- Once a transaction commits, its updates persist permanently in secondary storage and will survive subsequent power failures, crashes, or OS reboots via Write-Ahead Logging (WAL) and checkpointing.

Q10 (15M) **What is Normalization? Explain 1NF, 2NF, 3NF, and BCNF with suitable relational examples and anomaly reduction.**

Definition: Normalization is a formal process of decomposing relations with anomalies into structured relations to eliminate insertion, deletion, and update anomalies while avoiding redundant data storage.

1. First Normal Form (1NF):

- A relation is in 1NF if and only if all attribute values are atomic (no multivalued, composite, or nested attributes).
- *Violation:* `Employee(EmpID, Skills)` where `Skills = "Java, Python, C++"`.
- *Decomposition:* Split into atomic rows: `(101, 'Java')`, `(101, 'Python')`, `(101, 'C++')`.

2. Second Normal Form (2NF):

- Relation must be in 1NF and have **no partial dependency** (no non-prime attribute should depend on a proper subset of any candidate key).
- *Example:* `Enrollment(StudentID, CourseID, CourseFee, Grade)`. Here `CourseFee` depends only on `CourseID` (partial).
- *Decomposition:* `StudentCourses(StudentID, CourseID, Grade)` and `CourseDetails(CourseID, CourseFee)`.

3. Third Normal Form (3NF):

- Relation must be in 2NF and have **no transitive dependency** (for every non-trivial FD X \to Y, either X is a super key or Y is a prime attribute).
- *Example:* `Employee(EmpID, EmpName, DeptNo, DeptHead)`. Here `EmpID → DeptNo` and `DeptNo → DeptHead`, creating transitive dependency.
- *Decomposition:* `Emp(EmpID, EmpName, DeptNo)` and `Dept(DeptNo, DeptHead)`.

4. Boyce-Codd Normal Form (BCNF):

- A stricter version of 3NF where for every non-trivial FD X \to Y, determinant X **must strictly be a Super Key**.

Q11 (15M) Explain SQL Aggregate Functions (COUNT, SUM, AVG, MIN, MAX) with GROUP BY and HAVING clauses. Provide full query demonstrations.

Aggregate Functions: Functions that perform calculations on a set of column values and return a single scalar summary value.

- `COUNT(column)` : Returns total count of non-null values. `COUNT(*)` counts all rows.
- `SUM(column)` : Calculates total numeric sum of values.
- `AVG(column)` : Calculates arithmetic mean of non-null values.
- `MIN(column)` / `MAX(column)` : Returns the minimum / maximum value.

Role of GROUP BY and HAVING:

- `GROUP BY` divides records into distinct groups based on identical column values.
- `HAVING` filters grouped rows after aggregation (whereas `WHERE` filters individual tuples prior to grouping).

Comprehensive SQL Example:

```
-- Schema: Employee (EmpID, EmpName, Salary, DeptID, City)
-- Query: Find DeptID, total employees, average salary, and max salary
-- for departments having more than 3 employees and average salary > 45000:

SELECT
    DeptID,
    COUNT(EmpID) AS TotalEmployees,
    AVG(Salary) AS AverageSalary,
    MAX(Salary) AS HighestSalary,
    MIN(Salary) AS LowestSalary
FROM Employee
WHERE City IN ('Delhi', 'Noida', 'Gurugram')
GROUP BY DeptID
HAVING COUNT(EmpID) > 3 AND AVG(Salary) > 45000
ORDER BY AverageSalary DESC;
```

Q12 (15M) **What is Relational Algebra? Discuss its fundamental operations (Select, Project, Union, Set Difference, Cartesian Product, Rename) and derived Join operation.**

Definition: Relational Algebra is a formal procedural query language that takes one or two relations as input and produces a new relation as output without modifying the underlying stored tables.

Fundamental Operators:

1. **Select (σ):** Unary operator that filters rows satisfying a predicate condition P.

```
σ_{Salary > 50000 AND Dept = 'IT'} (Employee)
```

2. **Project (π):** Unary operator that selects specified columns and eliminates duplicate tuples.

```
π_{EmpName, Salary} (Employee)
```

3. **Union (∪):** Binary operator combining tuples from two union-compatible relations.

```
Student_CS ∪ Student_IT
```

4. **Set Difference (−):** Returns tuples present in relation R but not in S (union compatible).

```
All_Students - Placed_Students
```

5. **Cartesian Product (×):** Cross product pairing every tuple of R with every tuple of S.

```
Employee × Department
```

6. **Rename (ρ):** Renames relation or attributes: \rho_{S(A_1, A_2)} (R).

Derived Operation — Natural Join (⋈): Combines tuples of two relations matching on identical common attribute names:

```
Employee ⋈ Department = π_{...} (σ_{Employee.DeptID = Department.DeptID} (Employee × Department))
```

Q13 (15M) **Draw an ER Diagram for a Hospital Management System specifying entities, attributes, primary keys, relationships, and cardinalities. Show conversion to Relational Schema.**

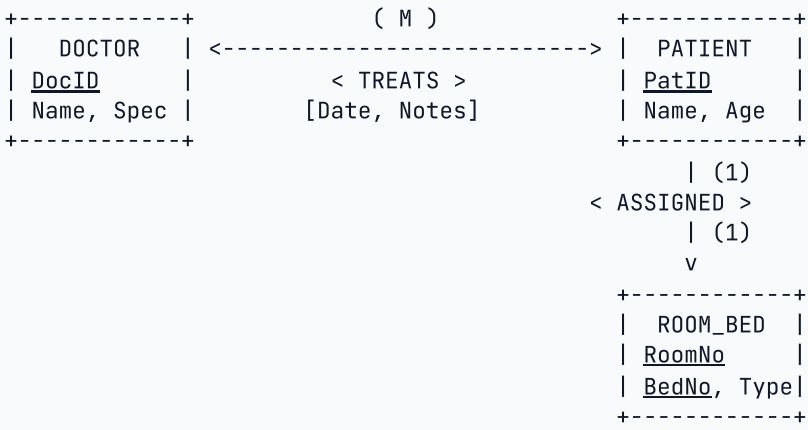
1. Entity Sets & Attributes:

- **DOCTOR**: DocID, Name, Specialty, Phone
- **PATIENT**: PatID, Name, Age, Gender, Address
- **ROOM / BED**: RoomNo, BedNo, RoomType, DailyCharge
- **BILL / ACCOUNT**: BillID, BillDate, TotalAmount, PaymentStatus

2. Relationships & Cardinalities:

- `TREATS` : Many-to-Many between DOCTOR and PATIENT with attribute `DateOfTreatment` .
- `ASSIGNED_TO` : Many-to-One between PATIENT and BED (A patient is assigned to one bed; a bed holds one patient at a time).
- `GENERATES` : One-to-One between PATIENT and BILL.

3. ASCII ER Diagram Representation:



4. Relational Schema Reduction:

1. Doctor(DocID, Name, Specialty, Phone)
2. Patient(PatID, Name, Age, Gender, Address, RoomNo, BedNo)
   FOREIGN KEY (RoomNo, BedNo) REFERENCES Room_Bed(RoomNo, BedNo)
3. Room_Bed(RoomNo, BedNo, RoomType, DailyCharge)
4. Treats(DocID, PatID, TreatmentDate, Prescription)
   FOREIGN KEY (DocID) REFERENCES Doctor(DocID),
   FOREIGN KEY (PatID) REFERENCES Patient(PatID)
5. Bill(BillID, PatID, BillDate, TotalAmount, Status)
   FOREIGN KEY (PatID) REFERENCES Patient(PatID)